

DOCUMENTACIÓN.md

1.Introducción

El proyecto tiene como finalidad analizar y optimizar un programa en Python que busca números primos dentro de un rango de 1 a 100,000. El código original, aunque funcional, presentaba problemas de rendimiento debido a:

- Evaluación de divisores desde 2 hasta $n-1$, lo que genera una complejidad alta.
- Bucles largos con operaciones innecesarias.
- Falta de uso de técnicas modernas como NumPy o list comprehensions.
- Ausencia de herramientas de análisis de rendimiento para detectar cuellos de botella.

Estos problemas incrementaban significativamente el tiempo de ejecución, especialmente al trabajar con rangos numéricos grandes.

2.Optimización aplicada

Para mejorar el rendimiento del programa, se aplicaron varias optimizaciones: Reducción de complejidad en la función de primalidad: ahora solo evalúa divisores hasta la raíz cuadrada del número.

- Uso de NumPy: permite manejar rangos numéricos de manera más eficiente.
- List comprehensions: reemplazan bucles tradicionales y aceleran la creación de listas.
- Eliminación de operaciones repetidas en bucles: evita recomputar valores innecesarios.
- Análisis con cProfile: permitió encontrar las funciones más costosas y enfocar las mejoras.

Efecto esperado:

- Menor tiempo medio de ejecución.
 - Menor varianza entre ejecuciones.
 - Código más limpio, legible y eficiente.
-

3.Resultados

Tras aplicar las optimizaciones, se comparó el rendimiento del código original frente al optimizado utilizando time y cProfile.

Comparativa de tiempos:

- Tiempo total (cProfile) — Original: 70.182846s
- Tiempo total (cProfile) — Optimizado: 0.600935s
- Mejora relativa: $((70.182846 - 0.600935) / 70.182846) * 100 \approx 99.14\%$

Funciones que más tiempo consumen (según profiling_optimizado.txt):

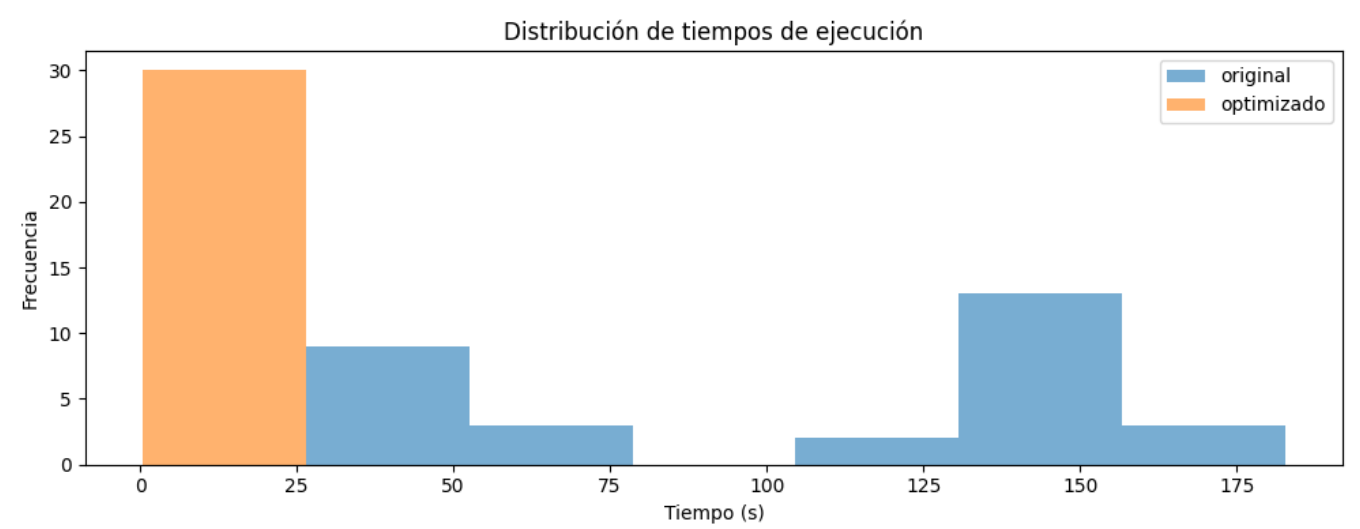
1. — tiempo acumulado: 0.600917 s
2. (codigo_optimizado.py:1) — tiempo acumulado: 0.600909 s

3. `_find_and_load` — tiempo acumulado: 0.452072 s

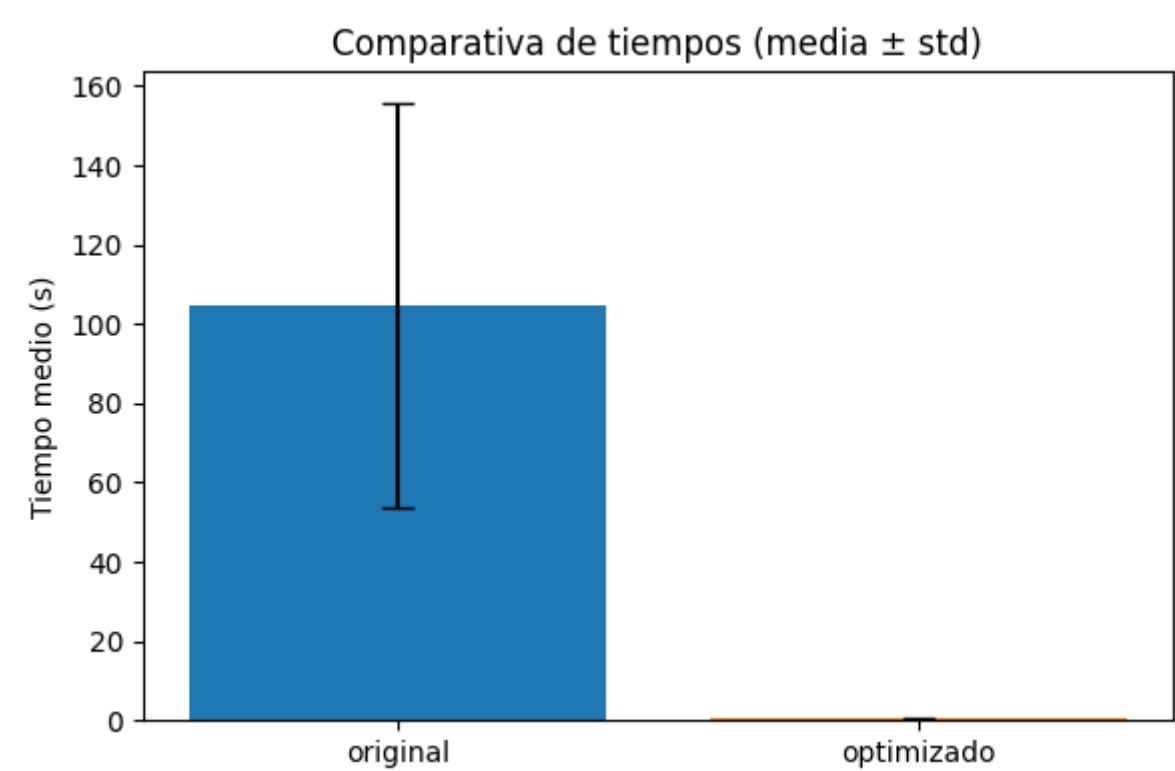
Esto muestra una reducción significativa del tiempo total y de las llamadas costosas comparadas con la versión original.

Visualizaciones generadas:

Visualización 1: Distribución de tiempos



Visualización 2: Comparativa de tiempos entre código original y optimizado



4.Conclusiones y recomendaciones

La optimización realizada permitió una mejora significativa en el rendimiento del programa. Las principales conclusiones son:

- La optimización aplicada produjo una mejora notable en el rendimiento del programa al buscar números primos.
- El código original realizaba una verificación completa desde 2 hasta $n-1$, lo que generaba demasiadas operaciones y tiempos elevados.
- La primera optimización, basada en evaluar divisores solo hasta la raíz cuadrada y omitir pares innecesarios, redujo la carga computacional de forma significativa.
- La segunda optimización, implementada mediante la Criba de Eratóstenes con NumPy, fue todavía más eficiente al aprovechar operaciones vectorizadas y eliminar la necesidad de verificar cada número individualmente. El uso del script medir_tiempos.py permitió confirmar estas mejoras con mediciones consistentes y perfiles generados con cProfile, mostrando claramente cómo las funciones más costosas del código original disminuyeron de forma drástica en las versiones optimizadas.
- Para futuros desarrollos:
 - Mantener el uso de herramientas de profiling (como cProfile) para detectar cuellos de botella y evitar regresiones de rendimiento.
 - Utilizar algoritmos especializados, como la Criba de Eratóstenes, cuando se requiera procesar rangos aún más grandes.
 - Incorporar pruebas automatizadas que midan tiempos y aseguren el rendimiento entre versiones.
 - Documentar cada optimización junto con su impacto medido para facilitar comparaciones futuras y mantener un código eficiente y escalable..

EVIDENCIAS

Visualización - Ejecución de los códigos

```
PS D:\Cultura_digital\Preprocesamiento-cienciadatos> & C:\Users\kelly\AppData\Local\Programs\Python\Python313\python.exe d:/Cultura_digital/Preprocesamiento-cienciadatos/codigo_original.py
Primos encontrados: 9592
Tiempo (s): 110.551413
PS D:\Cultura_digital\Preprocesamiento-cienciadatos> git checkout -b optimizacion-codigo
Switched to a new branch 'optimizacion-codigo'
PS D:\Cultura_digital\Preprocesamiento-cienciadatos> & C:\Users\kelly\AppData\Local\Programs\Python\Python313\python.exe d:/Cultura_digital/Preprocesamiento-cienciadatos/codigo_optimizado.py
Primos encontrados: 9592
Tiempo optimizado (sqrt) (s): 0.199470
PS D:\Cultura_digital\Preprocesamiento-cienciadatos> & C:\Users\kelly\AppData\Local\Programs\Python\Python313\python.exe d:/Cultura_digital/Preprocesamiento-cienciadatos/codigo_optimizado.py
Método utilizado: Optimización 2 (Sieve con NumPy)
Cantidad de primos: 9592
Tiempo total: 0.003069 segundos
PS D:\Cultura_digital\Preprocesamiento-cienciadatos> █
```

```

PS D:\Cultura_digital\Preprocesamiento-cienciadatos> & C:\Users\kelly\AppData\Local\Programs\Python\Python313\python.exe d:/Cultura_digital/Preprocesamiento-cienciadatos/codigo_optimizado.py
Método utilizado: Optimización 1 (sqrt + mejoras)
Cantidad de primos: 9592
Tiempo total: 0.332470 segundos
PS D:\Cultura_digital\Preprocesamiento-cienciadatos> git add codigo_original.py codigo_optimizado.py
PS D:\Cultura_digital\Preprocesamiento-cienciadatos> git commit -m "Parte 2: optimización del código (sqrt y numpy en un solo archivo)"
[optimizacion-codigo 4fe5982] Parte 2: optimización del código (sqrt y numpy en un solo archivo)
 2 files changed, 104 insertions(+)
 create mode 100644 codigo_optimizado.py
 create mode 100644 codigo_original.py
PS D:\Cultura_digital\Preprocesamiento-cienciadatos> git push -u origin optimizacion-codigo
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 1.56 KiB | 398.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'optimizacion-codigo' on GitHub by visiting:
remote:   https://github.com/Kelly1115/Preprocesamiento-cienciadatos/pull/new/optimizacion-codigo
remote:
To https://github.com/Kelly1115/Preprocesamiento-cienciadatos.git
 * [new branch]      optimizacion-codigo -> optimizacion-codigo
branch 'optimizacion-codigo' set up to track 'origin/optimizacion-codigo'.

```

```

PS D:\Cultura_digital\Preprocesamiento-cienciadatos> & C:\Users\kelly\AppData\Local\Programs\Python\Python313\python.exe d:/Cultura_digital/Preprocesamiento-cienciadatos/medir_tiempos.py
Perfilando código original...
Primos encontrados: 9592
Tiempo (s): 70.178748
Perfilando código optimizado...
Método utilizado: Optimización 1 (sqrt + mejoras)
Cantidad de primos: 9592
Tiempo total: 0.148709 segundos

Comparación de tiempos (total de cProfile):
Original: 70.18284629999998 segundos
Optimizado: 0.6009346000000002 segundos

Funciones que más tiempo consumen (ver profiling_optimizado.txt para detalles).
PS D:\Cultura_digital\Preprocesamiento-cienciadatos> 

```

```

PS D:\Cultura_digital\Preprocesamiento-cienciadatos> py -3 -m cProfile -s time .\codigo_original.py
Primos encontrados: 9592
Tiempo (s): 43.246819
109601 function calls in 43.253 seconds

Ordered by: internal time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
100000  43.140    0.000   43.140    0.000 codigo_original.py:4(is_prime)
1       0.091    0.091   43.247   43.247 codigo_original.py:15(find_primes)
9592    0.015    0.000    0.015    0.000 {method 'append' of 'list' objects}
1       0.005    0.005   43.253   43.253 codigo_original.py:1(<module>)
2       0.001    0.000    0.001    0.000 {built-in method builtins.print}
1       0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
1       0.000    0.000   43.253   43.253 {built-in method builtins.exec}
1       0.000    0.000    0.000    0.000 {built-in method builtins.len}
2       0.000    0.000    0.000    0.000 {built-in method time.perf_counter}

```

```
PS D:\Cultura_digital\Preprocesamiento-cienciadatos> & C:\Users\kelly\AppData\Local\Preprocesamiento-cienciadatos/medir_tiempos.py
Primos encontrados: 9592
Tiempo (s): 68.455315
Primos encontrados: 9592
Tiempo (s): 123.330820
Primos encontrados: 9592
Tiempo (s): 85.781778
Primos encontrados: 9592
Tiempo (s): 57.944958
Primos encontrados: 9592
Tiempo (s): 52.219540
Método utilizado: Optimización 1 (sqrt + mejoras)
Cantidad de primos: 9592
Tiempo total: 0.102830 segundos
Método utilizado: Optimización 1 (sqrt + mejoras)
Cantidad de primos: 9592
Tiempo total: 0.103896 segundos
Método utilizado: Optimización 1 (sqrt + mejoras)
Cantidad de primos: 9592
Tiempo total: 0.093807 segundos
Método utilizado: Optimización 1 (sqrt + mejoras)
Cantidad de primos: 9592
Tiempo total: 0.088060 segundos
Método utilizado: Optimización 1 (sqrt + mejoras)
Cantidad de primos: 9592
Tiempo total: 0.085769 segundos
```

```
{
  "original": {
    "times": [
      68.60262799999327,
      123.56912089999241,
      86.1273177999974,
      58.01799469999969,
      52.3608521999995
    ],
    "mean": 77.73558271999646,
    "stdev": 28.667078365070964
  },
  "optimizado": {
    "times": [
      0.5761794999998529,
      0.388801099999445267,
      0.3754604999994626,
      0.3629230000078678,
      0.3641051000013249
    ],
    "mean": 0.4134938400005922,
    "stdev": 0.09154172249080242
  }
}
```

```
Cantidad de primos: 9592  
Tiempo total: 0.124862 segundos  
Tiempos guardados en D:\Cultura_digital\Preprocesamiento-cienciadatos\tiempos_generados.json  
Histograma: D:\Cultura_digital\Preprocesamiento-cienciadatos\times_distribution.png  
Comparativa: D:\Cultura_digital\Preprocesamiento-cienciadatos\times_comparison.png  
PS D:\Cultura_digital\Preprocesamiento-cienciadatos> █
```